Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Asıl ders yaklaşık 00:07'de başlar. Kaydın başı bekleme kısmıdır.
- Önceki konulara dair öğrenci soruları
  - Interface sorusu (soru 3): adres hattı numarasındaki bir hata düzeltildi.
  - Seri iletişim çerçevesi: hoca önceki derste çerçeveyi yanlışlıkla "stop" ile başlattığını söyledi ve düzeltti.
- Cache memory (önbellek)
  - Bellek hiyerarşisi (memory hierarchy): register, L1 cache, L2 cache, ana bellek (DRAM), disk.
  - Yerellik ilkesi (principle of locality): zamansal (temporal) ve uzamsal (spatial) yerellik.
  - İşlemci ile DRAM arasındaki hız farkı (processor-memory gap).
  - Direct mapped cache: adresin tag, index, word offset ve byte offset alanlarına bölünmesi.
  - Adım adım direct mapped cache örneği: 0x14, 0x1C, 0x34, 0x8014 adreslerinin okunması.
  - Valid bit, cache hit, cache miss ve blok yerine koyma (replacement).
  - Ortalama erişim süresi (average access time) ve iki seviyeli cache örneği.
  - Yerine koyma algoritmaları: LRU, FIFO, random, optimal.
  - Yazma politikaları: write-through ve write-back, dirty bit.
  - Bitiş örneği: 32 bit adres, 64 bit veri yolu, 2 MB cache, 32 byte line için fully associative, direct mapped ve 2-way set associative alan hesapları.

Hocanın Özellikle Vurguladığı Kısımlar

- Dersin son dört haftası en önemli konuları içerir.
  - Bunlar giriş/çıkış ve iletişim, cache memory ve virtual memory konularıdır.
  - Bu konular Mikroişlemciler dersinde de işinize yarayacak.
- Cache konusunda ezber gerekmez.
  - Matematiği dikkatli takip ederseniz her soruyu çözebilirsiniz.
- Word, veri yolu genişliği kadardır.
  - 32 bit veri yolunda 1 word = 4 byte. 64 bit veri yolunda 1 word = 8 byte.
  - 8086'da öğrendiğiniz "1 word = 16 bit" bilgisini buraya taşımayın.
- Cache kapasitesi yalnız veri (data) kapasitesidir.
  - Tag, valid ve dirty bitleri kapasite hesabına katılmaz.
- Cache'te adresin tamamı değil, yalnız tag alanı saklanır.
  - Index ve offset alanları satırın yerinden ve satır içindeki konumdan zaten bellidir.
- Hit için iki koşul gerekir: valid bit 1 olmalı ve tag'ler eşleşmeli.
  - Valid bit 1 olduğu hâlde tag tutmazsa sonuç miss'tir.
- Direct mapped cache'te seçim yoktur.
  - Index hangi satırı gösteriyorsa blok oraya yazılır. Yerine koyma politikası yalnız associative ve set associative yapılarda anlamlıdır.
- Seri iletişimde çerçeve her zaman start biti (lojik 0) ile başlar.
  - Stop biti lojik 1'dir. Stop bitlerinin sayısı 1, 1.5 veya 2 olabilir.

Kısa Tekrar Notları

- Bellek hiyerarşisinde işlemciden uzaklaştıkça kapasite artar, hız düşer, byte başına maliyet azalır.
- Zamansal yerellik: aynı adrese yakın zamanda tekrar erişilir (döngüler, alt programlar).
- Uzamsal yerellik: yakın adreslere yakın zamanda erişilir (diziler, ardışık komutlar).
  - Cache'e tek byte değil blok taşınmasının sebebi uzamsal yerelliktir.
- Direct mapped adres bölümü (yüksek bitten düşüğe): tag | index | word offset | byte offset.
  - Satır sayısı $= \dfrac{\text{cache kapasitesi}}{\text{blok boyutu}}$, index bit sayısı $= \log_2(\text{satır sayısı})$.
  - Byte offset $= \log_2(\text{word içindeki byte sayısı})$, word offset $= \log_2(\text{bloktaki word sayısı})$.
  - Tag = adres genişliği − index − offset.
- Ortalama erişim süresi: $T_a = H\,T_c + (1-H)(T_c + T_m) = T_c + (1-H)\,T_m$.
- İki seviyeli cache: L1 miss cezası $= T_{L2} + m_{L2} \cdot T_{mem}$; AMAT $= T_{L1} + m_{L1} \cdot (\text{L1 miss cezası})$.
- Write-through: her yazmada cache ve ana bellek birlikte güncellenir; bellek her zaman tutarlıdır.
- Write-back: yalnız cache güncellenir, dirty bit 1 yapılır; blok çıkarılırken dirty ise ana belleğe yazılır.
- Okuma işlemleri toplam erişimlerin yaklaşık %75'idir, yazma %25'idir (dersteki istatistik).

Detaylı Açıklamalar

Önceki derslere dair düzeltmeler

- Interface sorusu (soru 3):
  - 8 bitlik adres yolu var. En yüksek iki bit (A7, A6) bütün interface'lerin register select uçlarına ortak bağlanır.
  - Kalan altı bit (A0–A5) altı interface'in chip select uçlarına tek tek bağlanır: A0 birinci interface'e, A5 altıncı interface'e.
  - Kitaptaki soru metninde "address line 4" yazıyordu. Öğrenci bunun 5 olması gerektiğini gösterdi, hoca onayladı: kitap da hata yapabilir.
  - Her interface'in 4 iç register'ı olduğu için her interface I/O alanında 4 adres kaplar. Soruda bu adresler isteniyor.
- Asenkron seri çerçeve:
  - Çerçeve bir start biti (lojik 0) ile başlar, veri bitleri gelir, bir veya daha fazla stop biti (lojik 1) ile biter.

Bellek hiyerarşisi

- CPU yongasının içinde register'lar ve L1 cache bulunur. Slayttaki eski mimaride L2 cache anakart üzerindedir.
- Slayttaki örnek değerler (eski bir mimari, mertebeler önemli):
  - Register erişimi yaklaşık 1.4 ns, L1 yaklaşık 4.2 ns, L2 bunun yaklaşık 4 katı.
  - Ana bellek L2'den yaklaşık 5 kat yavaş. Disk en yavaş ve en ucuz seviye.
- İşlemci-bellek hız farkı:
  - Derste anlatılan: işlemci hızı yılda yaklaşık %60 artarken DRAM erişim hızı yılda yalnız yaklaşık %9 iyileşiyor. Aradaki açık her yıl büyüyor.
  - Çözümlerden biri veri yolunu genişletmek (16 → 32 → 64 bit). Diğeri cache kullanmak.

Direct mapped cache: alan hesabı

- Örnek: 32 bit mimari (adres ve veri yolu 32 bit), 16 KB veri kapasiteli direct mapped cache, blok = 4 word.
  - 1 word = 4 byte, 1 blok = 16 byte $= 2^4$ byte.
  - Satır sayısı $= 2^{14}/2^4 = 2^{10}$, index 10 bit.
  - Byte offset 2 bit, word offset 2 bit, offset toplam 4 bit.
  - Tag $= 32 - 10 - 4 = 18$ bit.

- Blok boyutunu büyütmek miss oranını bir noktaya kadar düşürür.
  - Daha büyük bloklarda iyileşme durur, belli bir noktadan sonra miss oranı tekrar artar. Hoca bunu cache kapasitesinin sınırlarının zorlanmasına bağladı.
  - Ek bilgi (derste anlatılmadı): Sabit kapasitede blok büyüdükçe satır sayısı azalır.

Direct mapped cache: adım adım örnek

- Bellekte 0x10, 0x14, 0x18, 0x1C adreslerinde a, b, c, d; 0x30–0x3C adreslerinde e, f, g, h vardır.
- İşlemci sırayla 0x14, 0x1C, 0x34, 0x8014 adreslerinden okur. Başta bütün valid bitleri 0'dır.

Adres	Tag	Index	Word offset	Sonuç
0x14	0	1	1	Miss (valid = 0). a, b, c, d bloğu 1. satıra yüklenir, b okunur.
0x1C	0	1	3	Hit (valid = 1, tag eşit). d okunur.
0x34	0	3	1	Miss (valid = 0). e, f, g, h bloğu 3. satıra yüklenir, f okunur.
0x8014	2	1	1	Miss (valid = 1 ama tag 0 ≠ 2). 1. satırdaki blok yenisiyle değiştirilir.

- 0x1C'nin hit olması uzamsal yerelliğin etkisidir: 0x14 okunurken bütün blok gelmişti.
- 0x8014 bir çakışma (conflict) miss'idir. Direct mapped yapıda o bloğun gidebileceği tek satır 1. satırdır.

Ortalama erişim süresi

- $T_a = T_c + (1 - H)\, T_m$.
  - Miss durumunda önce cache'e bakılır, sonra ana belleğe gidilir. Bu yüzden miss süresi $T_c + T_m$ olarak alınır.
- Örnek: $H = 0.85$, $T_c = 5$ ns, $T_m = 50$ ns.
  - $T_a = 5 + 0.15 \cdot 50 = 12.5$ ns. Cache olmadan 50 ns olurdu.
- İki seviyeli örnek: L1 hit süresi 1 cycle, L1 miss oranı %5; L2 hit süresi 5 cycle, L2 miss oranı %15; ana bellek 100 cycle.
  - L1 miss cezası $= 5 + 0.15 \cdot 100 = 20$ cycle.
  - AMAT $= 1 + 0.05 \cdot 20 = 2$ cycle.

Yazma politikaları ve LRU

- Write-through:
  - Her yazmada veri hem cache'e hem ana belleğe yazılır. Bellek ve cache her zaman tutarlıdır.
  - Bedeli: her yazma ana bellek trafiği üretir.
- Write-back:

  - Yazma yalnız cache'e yapılır, satırın dirty biti 1 olur.
  - Satır başka bir blokla değiştirileceği zaman dirty biti 1 ise önce ana belleğe yazılır.
- Fully associative örnekte (2 satırlık cache) LRU biti her erişimde güncellenir.
  - Yeni blok, boş satıra ya da en uzun süredir kullanılmayan satıra yerleşir.
  - Örneğin devamı (write-back sürümü dahil) öğrencilere bırakıldı.

Bitiş örneği: üç eşleme tekniğinde alanlar

- Verilenler: adres 32 bit, veri yolu 64 bit (1 word = 8 byte), cache 2 MB (tag hariç), line = 32 byte = 4 word.
- Byte offset $= \log_2 8 = 3$ bit, word offset $= \log_2 4 = 2$ bit.

Teknik	Tag	Index	Word offset	Byte offset
Fully associative	27	—	2	3
Direct mapped	11	16	2	3
2-way set associative	12	15	2	3

- Direct mapped: satır sayısı $= 2^{21}/2^5 = 2^{16}$, index 16 bit.
- 2-way: her set yarı kapasiteyi alır. Satır sayısı 2^{15}, index 15 bit, tag 1 bit artar.

Ders sonrası notlar

- Hoca interface sorusunu anlatırken "A1 ucu birinci interface'in chip select ucuna bağlanacak" dedi, ikinci interface için de A1 dedi. Doğrusu, kendisinin bundan önce söylediği gibi, A0 birinci, A1 ikinci interface'e bağlanır.
- Gordon Moore derste "Intel'de çalışan bir iktisatçı, işletmeci" diye anlatıldı.
  - Moore bir kimyagerdir ve Intel'in kurucularındandır.
  - Moore Yasası, bir yongadaki transistör sayısının yaklaşık her iki yılda bir ikiye katlanmasıdır.
  - "İşlemci yılda %60, DRAM yılda yaklaşık %9 hızlanıyor" karşılaştırması Moore Yasası değildir. Bu, Hennessy & Patterson'ın işlemci-bellek performans açığı (processor-memory gap) grafiğidir.
- Hoca LRU'yu anlatırken "direct map cache'te ne yapıyorduk? Aynı indekse sahip dört set var" dedi. Bu cümlede kastedilen set associative cache'tir. Direct mapped yapıda her index için tek satır vardır.
- Slayttaki bellek hiyerarşisi şekli eski bir mimariye aittir. Günümüz işlemcilerinde L2 (çoğunlukla L3 de) işlemci yongasının içindedir.
- Miss durumunda bloğun cache'e yazılmasının ayrıca süre eklemediği söylendi. Bu, dersteki basit modelin varsayımıdır. Gerçek sistemlerde blok yükleme (ve write-back'te dirty bloğun geri yazılması) miss cezasının bir parçasıdır.
- Slayttaki sürelerde L1 için "3 katı, 4 katı" dendi. $4.2/1.4 = 3$ kat.
- 64 bit veri yolu için "4 byte'ı bir seferde okuyorsunuz" dendi. 64 bit 8 byte'tır.
- LRU örneğinde 7 adresi için "tag 0111, adres bilgisinin tamamı tag" dendi. Blokta iki sayı olduğu için adresin son biti offset'tir: tag 011, offset 1.
- Bitiş örneğinde "32 MB, 2 üzeri 5" dendi. Doğrusu line 32 byte $= 2^5$ byte'tır.
- 2-way çözümünde önce "tag 11 bit" dendi, sonra 12 bit bulundu. Doğrusu 12 bittir.
- Tüm sayısal sonuçlar (12.5 ns, 20 ve 2 cycle, 0x8014 için tag 2 ve index 1, bitiş tablosu) Python ile doğrulandı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi daha verimli çalışabilirsiniz.